TP6 - Continuous Delivery de PhotoMatch

1. Objetivo y teoría

CI integra y verifica; Continuous Delivery deja un cambio verificado listo para producción con decisión humana; Continuous Deployment elimina ese gate. Un artefacto es la imagen que el pipeline produce. El tag **sha-<commit>** relaciona imagen y código. Un environment de GitHub limita secrets, puede exigir aprobación y registra deployments.

2. Situación inicial

TP5 tenía 32 tests y cobertura exigida. En PR #18 las imágenes backend/frontend se publicaban desde jobs paralelos. Uno podía publicar aunque el otro fallara. No existían entornos, secrets ni runner. No había cuentas de Render/Neon, por lo que se eligió el fallback local que autoriza la cátedra.

3. Archivos modificados

.github/workflows/ci.yml, **deploy/compose.yml**, **scripts/deploy-local.sh**, **scripts/smoke.sh**, **backend/Dockerfile**, **backend/src/config/database.js**, **backend/src/app.js**, **frontend/Dockerfile** y **frontend/default.conf.template**.

4. Código importante

publish usa **needs: [build-backend, build-frontend]**. Los builds exportan archivos de imagen; luego **docker load** y **docker push** publican esos mismos bytes sólo en **main**. QA y PROD ejecutan **image:** del mismo SHA mediante proyectos Compose separados. El smoke consulta **/health** repetidamente y compara **commit** con el SHA de la corrida. **/health** hace **SELECT 1**; un HTTP 200 de una versión vieja falla. **BACKEND_URL** llega a Nginx al arrancar. **DB_SSL_MODE=require** valida el certificado para un Postgres externo; el local usa **disable**.

5. Decisiones y alternativas

El fallback local evita cuentas de nube y permite una demostración reproducible. El runner propio necesita Docker Desktop y la máquina encendidos. Se eligió **recreate** por simplicidad y costo; para una producción con usuarios convendría rolling con capacidad duplicada y métricas. No se usa **latest** porque perdería la relación con el commit. QA se despliega sola; producción tiene required reviewer.

6. Herramientas

GitHub Actions ordena jobs con **needs**; GHCR almacena imágenes; GitHub Environments controla secretos y revisión; Docker Compose separa red y volumen por proyecto; Nginx enruta a Express; PostgreSQL persiste; ShellCheck y actionlint validan scripts y workflow.

7. Tests y evidencia real

Localmente: 32 unitarios, 100% de líneas y 70,58% de ramas; builds de ambas imágenes; Compose con **/health** y **SELECT 1**; **actionlint** y ShellCheck. PR #18 sigue abierto. **Faltan** corrida de **main** publicando paquetes públicos, QA/PROD desde el runner, rechazo y aprobación, rollback medido y **v6.0.0**.

8. Problemas y solución

La publicación paralela violaba el gate global. Se separó verificación de publicación y se transportan los artefactos. El certificado PostgreSQL se validaba con **rejectUnauthorized:false**; ahora se verifica. El secret de sesión de ejemplo podía usarse en producción; ahora el arranque lo rechaza.

9. Preguntas del profesor

¿Qué garantiza el registry? En este workflow, un push de **main** sólo publica tras ambos jobs verdes. Un push manual fuera del pipeline no queda cubierto. **¿Qué prueba el smoke?** Salud de app, conexión a DB y SHA; no prueba crear un cliente. **¿Qué revisa el aprobador?** Unitarios, cobertura, imágenes, smoke, integración, E2E y el diff. **¿Cómo harías rollback?** Reusar los dos tags anteriores y medir hasta que **/health** responda con ese SHA; aún falta ejecutarlo.

10. Demo de defensa

Abrir PR #18, la corrida PR con publicación salteada, la corrida de **main** con publicación al final y GHCR. Mostrar los dos proyectos Compose y **/health** en puertos 3001/3002, los environments y el gate de PROD. Enseñar corrida rechazada y luego aprobada, más el tiempo real del rollback cuando existan. No presentar los pendientes como hechos.